

Helix OTA — Performance / Scaling Baseline (1.0.0-MVP)

Revision: 1 **Last modified:** 2026-06-08T21:00:00Z **Authority:** Measured baseline produced by `server/tools/loadtest` (stdlib black-box harness) **Scope:** Control-plane HTTP server NFR baseline — `cmd/ota-server`, in-memory and `pgx/PostgreSQL` backends

1. Purpose and anti-bluff statement (Constitution §11.4)

Every number in this document is **MEASURED** from real HTTP request/response round-trips against a live `ota-server`, captured by the `server/tools/loadtest` stdlib black-box harness. **These are measured baselines on the host described in §2 — NOT production NFR guarantees.** No NFR target is asserted here; the harness reports only what it observed (see `server/tools/loadtest/README.md`). Comparison of these measurements against any stated NFR target is a separate exercise.

The host was **busy during the run** (load average 16.59 on 11 cores: a concurrent `:8080` job plus the load generator co-resident with the server under test). These numbers are therefore **conservative, contention-affected** baselines, not the server’s best-case ceiling.

Raw harness JSON + stderr tables for every run are committed under `docs/qa/20260608-perf/(<backend>_<path>_c<N>.json / .txt, harness_selftest.{json,txt}, run_conditions.txt)`.

2. Host / run conditions

Property	Value
Host CPU	Apple M3 Pro, 11 cores
Host RAM	18 GB
OS	macOS 15.5 (build 24F74)
Go toolchain	go1.26.2 darwin/arm64
Host load avg during run	16.59 / 20.75 / 17.94 (busy)
Server	go run ./cmd/ota-server, plain HTTP on <code>:8090</code> , Gin ReleaseMode, base path <code>/api/v1</code>
Harness	

Property	Value
	server/tools/loadtest → /tmp/ loadtest (stdlib only; go vet clean; selftest validated)
Sweep	-concurrency {1, 8, 32, 128}, -duration 15s per level
In-memory backend	default (no HELIX_DATABASE_URL)
pgx backend	postgres:16-alpine in podman (applehv VM, 2 CPU / 4 GiB) on :55432

Probe paths. Two unauthenticated GET endpoints were swept (all other GETs require a JWT the black-box harness does not mint):

- /healthz — pure handler, touches no store. Measures HTTP/router/runtime cost.
 - /readyz — calls `repo.GetIdempotent(...)`, a real store round-trip.
- Representative store-touching GET** — the path where the in-memory vs pgx backend difference shows up.

On the “Errors” column. Every run reports `errors == concurrency` and `non_2xx == 0`. Those errors are the **expected context-cancel-at-duration-boundary** events (one in-flight request per worker when the 15s deadline fires) — they are a harness end-of-test artifact, **not** server failures. Zero non-2xx responses were observed in any run.

3. In-memory backend — measured

3.1 /healthz (no store)

Concurrency	p50 (ms)	p90 (ms)	p99 (ms)	RPS	Total req	Server RSS (MB)
1	0.238	0.696	2.090	2,730.3	40,955	29.4
8	0.775	2.208	10.095	5,165.6	77,752	258.0
32	3.277	14.217	174.913	3,226.7	48,401	345.6
128	5.905	21.168	46.039	14,002.7	210,076	330.7

The c=32 row’s 174.9ms p99 / depressed RPS is a transient host-scheduling jitter spike in that single 15s window (the host was at load avg ~16). The /readyz c=32 run (§3.2) under the same backend shows a normal 11.6ms p99, confirming the spike is host contention, not a server characteristic. Reported as-measured per §11.4.6 — not smoothed away.

3.2 /readyz (store round-trip) — representative GET

Concurrency	p50 (ms)	p90 (ms)	p99 (ms)	RPS	Total req	Server RSS (MB)
1	0.185	0.398	0.942	4,209.7	63,147	294.3
8	0.819	2.115	9.795	5,925.0	88,877	248.4
32	1.864	5.027	11.633	12,245.2	183,723	215.2
128	3.976	14.313	30.560	20,532.4	308,013	301.4

The in-memory /readyz path scales monotonically: RPS 4.2k → 5.9k → 12.2k → 20.5k as concurrency rises 1 → 8 → 32 → 128, with p99 staying ≤ 30.6ms.

4. pgx / PostgreSQL backend — measured

Postgres ran in a **2-CPU / 4-GiB podman VM** — the DB tier, not the server, is the bottleneck on the store-touching path. Server process RSS is materially lower than the in-memory backend (no in-process store).

4.1 /healthz (no DB)

Concurrency	p50 (ms)	p90 (ms)	p99 (ms)	RPS	Total req	Server RSS (MB)
1	0.160	0.328	0.603	5,182.4	77,738	31.7
8	0.583	1.121	2.042	12,129.3	181,940	195.9
32	1.727	4.355	8.412	14,638.6	219,584	196.1
128	4.713	16.963	40.796	16,988.7	254,844	260.9

4.2 /readyz (real pgx DB round-trip per request)

Concurrency	p50 (ms)	p90 (ms)	p99 (ms)	RPS	Total req	Server RSS (MB)
1	0.453	0.878	2.117	1,731.9	25,979	202.8
8	1.320	2.869	7.180	4,758.7	71,383	153.0
32	5.244	8.527	17.942	5,422.6	81,342	154.8
128	18.758	28.526	56.026	6,235.1	93,534	155.0

5. In-memory vs pgx comparison

Store-touching /readyz path (the meaningful comparison):

Concurrency	in-mem RPS	pgx RPS	in-mem p50 / p99 (ms)	pgx p50 / p99 (ms)	in-mem RSS	pgx RSS
1	4,209.7	1,731.9	0.185 / 0.942	0.453 / 2.117	294 MB	203 MB
8	5,925.0	4,758.7	0.819 / 9.795	1.320 / 7.180	248 MB	153 MB
32	12,245.2	5,422.6	1.864 / 11.633	5.244 / 17.942	215 MB	155 MB
128	20,532.4	6,235.1	3.976 / 30.560	18.758 / 56.026	301 MB	155 MB

Measured observations (this host only):

- **Throughput:** on the store-touching path the in-memory backend reaches $\sim 3.3\times$ the RPS of pgx at $c=128$ (20.5k vs 6.2k). pgx / readyz RPS plateaus near $\sim 6.2k$ — the **2-CPU containerized Postgres tier saturates**, while the pure-HTTP / healthz path is comparable across backends (16.9k pgx vs 14.0k in-mem at $c=128$).
- **Latency:** pgx / readyz p50 climbs to 18.8ms at $c=128$ (each request = one DB round-trip into the constrained VM) vs 4.0ms in-memory; pgx p99 56.0ms vs in-mem 30.6ms.
- **Memory:** the **pgx server process uses less RSS** ($\sim 153\text{--}203$ MB) than the in-memory server ($\sim 215\text{--}301$ MB), as expected — pgx pushes state into Postgres rather than holding it in-process. (Total system memory for the pgx topology is higher once the Postgres container itself is counted.)

6. Honest limitations (§11.4.6)

- Single host, single 15s window per concurrency level — no N-iteration determinism run (§11.4.50) was performed; treat individual rows as point samples. The $c=32$ in-mem / healthz spike is the clearest evidence of single-window jitter.
- Host was under heavy concurrent load (load avg ~ 16 on 11 cores), so these are contention-affected, conservative numbers — not the server's isolated ceiling.
- pgx numbers are bounded by a **2-CPU / 4-GiB** containerized Postgres, not a production DB tier; the pgx RPS plateau reflects that DB sizing, not a server limit.
- Only unauthenticated GETs (/healthz, /readyz) were swept; authenticated control-plane endpoints (JWT-gated) and the artifact-byte download path were not measured (the black-box harness mints no JWT and the download path is on a separate router). Those are follow-up work.
- Per §11.4: **these are MEASURED baselines on this host, not production NFR guarantees.**

7. Reproduction

```
cd server
go build -o /tmp/loadtest ./tools/loadtest/ # stdlib only

# in-memory
HELIX_PORT=8090 HELIX_ADMIN_USERNAME=admin@helix.test \
  HELIX_ADMIN_PASSWORD=s3cret HELIX_TOKEN_SECRET=perf \
  go run ./cmd/ota-server &
for c in 1 8 32 128; do
  /tmp/loadtest -url http://127.0.0.1:8090 -path /readyz
    -concurrency $c -duration 15s
done

# pgx (containerized Postgres on a free port)
podman run -d --name helix-perf-pg -e POSTGRES_PASSWORD=perf -e
  POSTGRES_USER=perf \
  -e POSTGRES_DB=helix -p 55432:5432 docker.io/library/
  postgres:16-alpine
HELIX_PORT=8090 HELIX_DATABASE_URL='postgres://
  perf:perf@127.0.0.1:55432/helix?sslmode=disable' \
  HELIX_ADMIN_USERNAME=admin@helix.test
  HELIX_ADMIN_PASSWORD=s3cret HELIX_TOKEN_SECRET=perf \
  go run ./cmd/ota-server &
# re-run the sweep, then: podman rm -f helix-perf-pg

Raw evidence: docs/qa/20260608-perf/.
```